

GBM8378 - Principes d'imagerie biomédicale ## Laboratoire 3 - Imagerie par Résonance Magnétique
 Zire Aymen - 2301311 **Rabhi Ayoub - 2131345**

Assurez vous d'avoir indiqué vos noms et matricules dans la cellule précédente.

Avant de commencer à répondre au différentes questions, veuillez vérifier que le Jupyter Notebook fonctionne correctement:

1. Redémarrez le noyau (dans la barre de menu : "Kernel" $\rightarrow$ "Restart")
2. Lancez toutes les cellules (dans la barre de menu : "Cell" $\rightarrow$ "Run All").

Travail à effectuer

****Avant de remettre votre travail, assurez-vous d'avoir rempli toutes les sections "Ajoutez votre code ici" et "Double-cliquez pour entrez votre réponse".****

```
In [2]: import math
import matplotlib.pyplot as plt
import nibabel as nib
import nilearn.plotting as nip
import numpy as np
from copy import deepcopy
from numpy.fft import *
from numpy.matlib import *
from numpy import matlib as mb
from niwidgets import NiftiWidget
import pydicom

%matplotlib inline
```

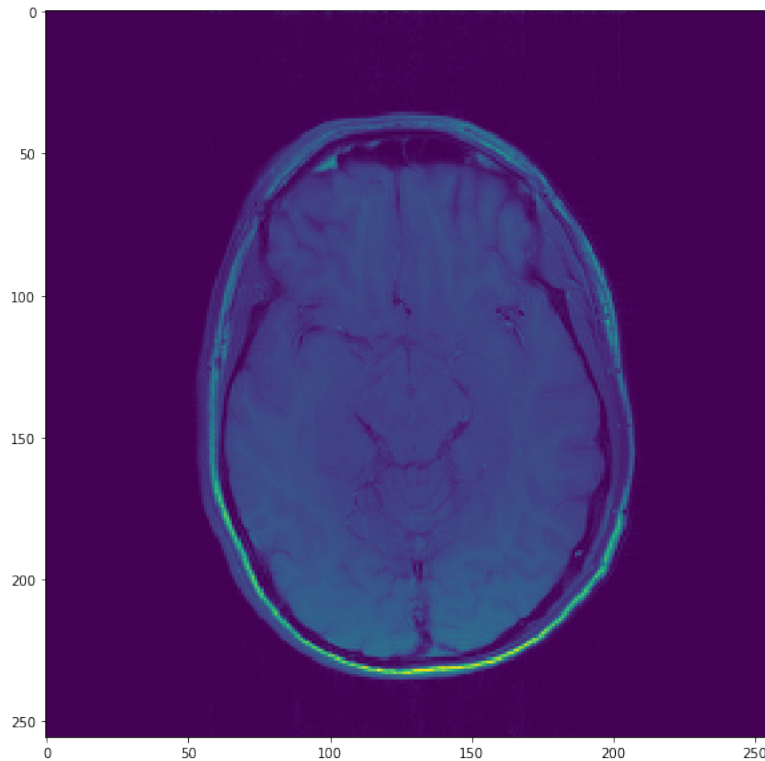
Section 1 - Concepts d'IRM (20pts)

****Lecture d'un fichier DICOM**** Les images issues du système IRM sont sous format DICOM (extension .dcm). DICOM est un standard de fichier en imagerie médicale (pas seulement pour l'IRM, mais aussi pour le PET, CT, US, etc...). Les fichiers DICOM incluent un header qui contient des informations ou "metadata" sur l'acquisition des images (e.g., date de l'examen, modèle d'IRM, TR, TE, flip angle, position des slices, etc...), suivi de l'image encodée en binaire. Une image 3D est sauvegardée sous plusieurs tranches, i.e. plusieurs fichiers DICOM. Ci-dessous, nous analyserons le contenu du fichier DICOM `DicomFile_magnitude.dcm`, notamment nous afficherons l'image et nous analyserons quelques champs des metadata. Pour plus d'informations sur les fichiers DICOM et les données qu'ils contiennent, vous pouvez consulter [ce site](https://dicom.innolitics.com/ciods).

```
In [3]: # La ligne ci-dessous sert à lire l'image DICOM et à la mettre dans la variable dcm
dcm = pydicom.dcmread('dicom/DicomFile_magnitude.dcm', force=True)

# L'attribut "pixel_array" sert à récupérer l'image (i.e. matrice 2d)
image = dcm.pixel_array

# Afficher l'image
plt.figure(figsize=(10,10))
plt.imshow(image)
plt.show()
```



1.1 Quelle est la largeur de la coupe (en z) ? Quelle est l'unité ? (1pt)

Pour afficher le contenu d'une variable en Python, vous pouvez utiliser la fonction ``print(dcm)``. Sélectionner ensuite le champ pertinent : ``dcm.CHAMP``

ATTENTION: Les espaces doivent être enlevés pour appeler le champ. Exemple: ``"Study Date" --> dcm.StudyDate``

```
In [23]: print("la largeur de la coupe en mm est de : ")
         print(dcm.SliceThickness)
```

```
la largeur de la coupe en mm est de :
5
```

La largeur de la coupe (Slice Thickness) est de 5 mm. L'unité est le millimètre car c'est le standard en imagerie médicale.

1.2 Étant donné que nous imageons des protons ^1H et que le champ est 3 teslas, estimez approximativement la valeur du gradient magnétique qui est imposé en z pour former une tranche de cette largeur si nous supposons que la largeur de bande de l'impulsion RF est de 1 kHz. (2pts)

```
In [6]: gamma = 42.58e6 # Hz/T
         delta_f = 1000 # Hz
         delta_z = 0.005 # metre (5mm = 0.005m)

         G = delta_f / (gamma * delta_z)
         print("Il faut un gradient de ", G, "T/m pour obtenir une largeur de 0.005m")
```

```
0.004697040864255519
```

La formule utilisée est $\Delta f = \gamma \cdot G \cdot \Delta z$, dérivée de l'équation de Larmor $f = \gamma B$ en présence d'un gradient magnétique. Le gradient en z est donc d'environ 0.0047 T/m ou 4.7 mT/m.

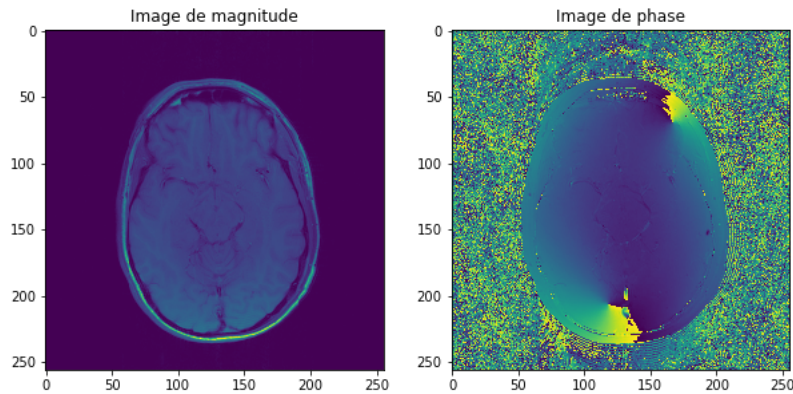
Analyse de Fourier

Rappelons que le signal acquis en IRM est en fait la transformée de Fourier de l'image (voir les équations 13.46 et 13.47 du livre de Prince & Links). L'image du cerveau qu'on affiche est donc la transformée inverse des données mesurées.

Le code ci-dessous permet d'ouvrir l'image de phase et d'afficher l'image de magnitude et de phase. Voir la [documentation](https://matplotlib.org/) pour plus d'informations.

```
In [15]: # On ouvre l'image de phase
dcm_phase = pydicom.read_file('dicom/DicomFile_phase.dcm', force=True)
image_phase = dcm_phase.pixel_array

# Display. Le package utilisé est matplotlib.pyplot qui a été importé en tant que plt. Nous appelons donc les fonction plt.fonction
plt.figure(figsize = (10, 10)) # plt.figure permet de décrire l'image. "figsize" détermine la taille de la figure.
plt.subplot(1, 2, 1) # Permet de créer un subplot. La numérotation est la suivante (nb_colonne,nb_lignes,index)
# L'indexation commence a 1 et va de gauche a droite et de haut en bas.
plt.imshow(image) # Permet d'afficher une image
plt.title('Image de magnitude') # Le titre s'applique a l'objet subplot, en l'occurrence le dernier existant.
plt.subplot(1, 2, 2)
plt.imshow(image_phase)
plt.title('Image de phase');
```



1.3. Les limitations d'acquisition liées au phénomène de Nyquist s'opèrent ici dans l'espace des fréquences spatiales. Pour étudier ce phénomène suivez les instructions ci dessous :

- Faites la transformée de Fourier 2D (fonction `fft2`) de l'image de magnitude `DicomFile\_magnitude.dcm`.
- Sous-échantillonnez cette transformée par un facteur 2 selon les deux axes (u et v). Pour conserver la taille de la matrice (256x256), remplacer un élément sur deux par des 0. Au final, ****vous devez avoir conservé la moitié des valeurs de la FFT originale.**** - Faites la transformée inverse. Illustrez l'image ainsi obtenue. Comparez-la avec l'image originale. Expliquez le phénomène observé et sa cause. (3pts)

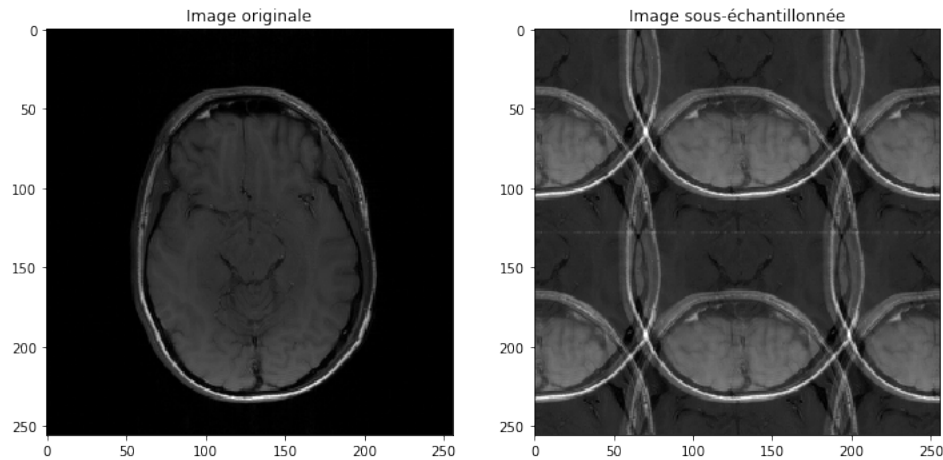
```
In [16]: # Transformée de Fourier 2D
fft_image = np.fft.fft2(image)

# Sous-échantillonnage (mettre 1 pixel sur 2 à 0)
fft_under = fft_image.copy()
fft_under[::2, ::2] = fft_image[::2, ::2]
fft_under[1::2, :] = 0
fft_under[:, 1::2] = 0

# Transformée inverse
image_recon = np.fft.ifft2(fft_under)

# Prendre la magnitude
image_recon = np.abs(image_recon)

plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.imshow(image, cmap='gray')
plt.title("Image originale")
plt.subplot(1,2,2)
plt.imshow(image_recon, cmap='gray')
plt.title("Image sous-échantillonnée")
plt.show()
```



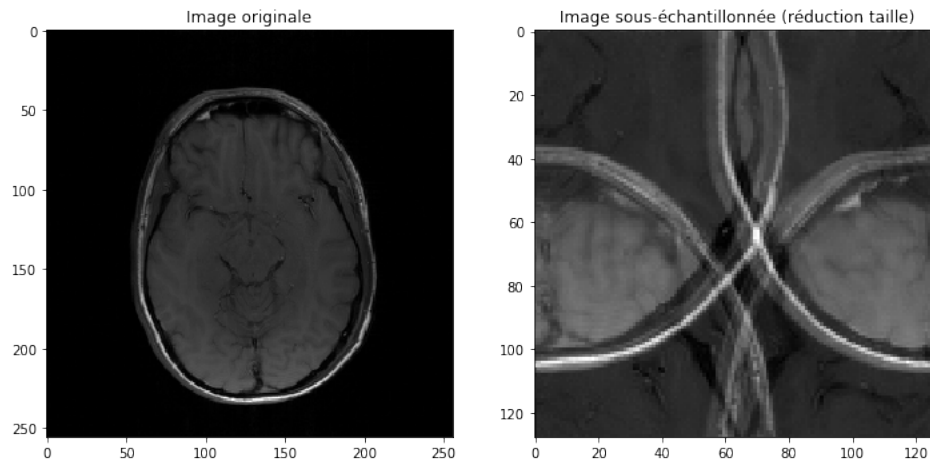
1.4. Recommencez avec l'image originale : - Faites sa transformée de Fourier 2D. - Sous-échantillonnez cette transformée par un facteur 2 selon les deux axes (u et v). Cette fois-ci, retirez un élément sur deux (en u et v, donc 1 ligne sur 2 et 1 colonne sur 2). Vous diminuerez donc de moitié la taille de votre matrice dans chaque dimension. - Faites la transformée inverse. Illustrez l'image ainsi obtenue. Comparez-la avec l'image originale et avec l'image reconstruite à la question 1.3. Expliquez les différences observées. (3pts)

```
In [17]: fft_image = np.fft.fft2(image)

# Sous-échantillonnage (on enlève des lignes/colonnes)
fft_under = fft_image[::2, ::2] # → taille divisée par 2 en x et y

# IFFT
image_recon = np.fft.ifft2(fft_under)
image_recon = np.abs(image_recon)

# Affichage
plt.figure(figsize=(12,6))
plt.subplot(1,2,1)
plt.imshow(image, cmap='gray')
plt.title("Image originale")
plt.subplot(1,2,2)
plt.imshow(image_recon, cmap='gray')
plt.title("Image sous-échantillonnée (réduction taille)")
plt.show()
```



Dans l'image originale, on observe clairement le cerveau avec une bonne résolution et sans artefacts. Dans la question 1.3, on voit apparaître un effet de sous-échantillonnage lié au critère de Nyquist. Ca se traduit par un phénomène d'aliasing, où l'image du cerveau est répétée plusieurs fois sous forme d'une mosaïque (9 images de cerveau qui se superposent). Ceci est dû au fait que des points du k-space ont été supprimés mais que la taille de la matrice a été conservée, ce qui entraîne un repliement spatial. Dans la question 1.4, l'image reconstruite est différente : elle apparaît comme zoomée et divisée en plusieurs parties, avec environ 4 portions du cerveau visibles qui se superposent. Contrairement à 1.3, il n'y a pas de mosaïque complète, mais plutôt une perte de champ de vue et de résolution. Cela est dû au fait que des lignes et colonnes ont été supprimées dans le k-space, réduisant la taille de l'image et la quantité d'information disponible.

1.5. Comment l'acquisition dans l'espace Fourier affecte la résolution spatiale de l'image ? Décrire l'équation qui régit la résolution spatiale en fonction des paramètres d'instrumentation de l'IRM. Quelles sont les contraintes d'instrumentation et de sécurité/confort du patient qui limitent la résolution spatiale ? (3pts)

En IRM, l'image est reconstruite à partir du k-space (espace de Fourier). La résolution spatiale dépend directement de l'étendue et de l'échantillonnage de cet espace. Plus on acquiert de hautes fréquences spatiales (k élevé), plus l'image est détaillée. La résolution spatiale est donnée approximativement par : $\Delta x \approx 1 / (2k_{\max})$ ou $\Delta x = \text{FOV} / N$, où FOV est le champ de vue et N le nombre de points échantillonnés. Le maximum de k dépend du gradient : $k_{\max} = (\gamma G T) / (2\pi)$. Donc, augmenter le gradient ou le temps d'acquisition permet d'améliorer la résolution. Cependant, plusieurs contraintes limitent la résolution spatiale. Du côté de l'instrumentation, les gradients sont limités en amplitude et en rapidité de variation, car les bobines ont des limites physiques et électriques, et des gradients trop forts peuvent dépasser les capacités du système. Du côté du patient, des gradients trop élevés peuvent induire des effets physiologiques comme une stimulation des nerfs périphériques, ainsi qu'un bruit acoustique important dû aux vibrations des bobines. Enfin, augmenter la résolution nécessite d'échantillonner davantage le k-space, donc d'augmenter le temps d'acquisition, ce qui rend l'image plus sensible aux mouvements du patient et peut donc dégrader la qualité finale de la reconstruction.

****Artéfacts**** Cette section a pour but de vous faire simuler 2 artéfacts qui peuvent survenir de manière assez fréquente lors de l'acquisition d'une image par résonance magnétique : artéfact de « spiking » et artéfact de mouvement.

1.6. **Artéfact de Spiking.** Décrivez une procédure permettant de simuler un artéfact « spiking » à partir d'une image déjà reconstruite. Comment cet artéfact affecte-t-il l'image finale ? (1.5pts)

Pour simuler un artéfact de spiking à partir d'une image déjà reconstruite, on agit dans l'espace de Fourier (k-space). Une procédure simple consiste à faire la transformée de Fourier 2D de l'image, puis à modifier artificiellement certaines valeurs en introduisant un ou plusieurs pics à des positions aléatoires ou spécifiques du k-space. Ensuite, on reconstruit l'image avec la transformée de Fourier inverse. Cet artéfact affecte l'image finale en introduisant des lignes ou motifs très brillants et fins qui traversent l'image. Ces structures viennent du fait qu'un point très intense dans le k-space influence globalement toute l'image après la transformée inverse, ce qui crée des rayons dans l'image reconstruite et dégrade sa qualité.

1.7. Implémentez cette procédure en utilisant `DicomFile.dcm` et simulez trois types de spiking artefacts : horizontal, vertical et oblique. (1.5pts)

```

In [19]: # FFT de l'image magnitude
kspace = np.fft.fft2(image)
kspace_shift = np.fft.fftshift(kspace)

def add_spike(kspace, pos, intensity=1e6):
    k = kspace.copy()
    x, y = pos
    k[x, y] = intensity
    return k

h, w = kspace_shift.shape
cx, cy = h//2, w//2

# Copies
k_h = kspace_shift.copy()
k_v = kspace_shift.copy()
k_o = kspace_shift.copy()

# Spike horizontal
k_h = add_spike(k_h, (cx, cy+20))
k_h = add_spike(k_h, (cx, cy-20))

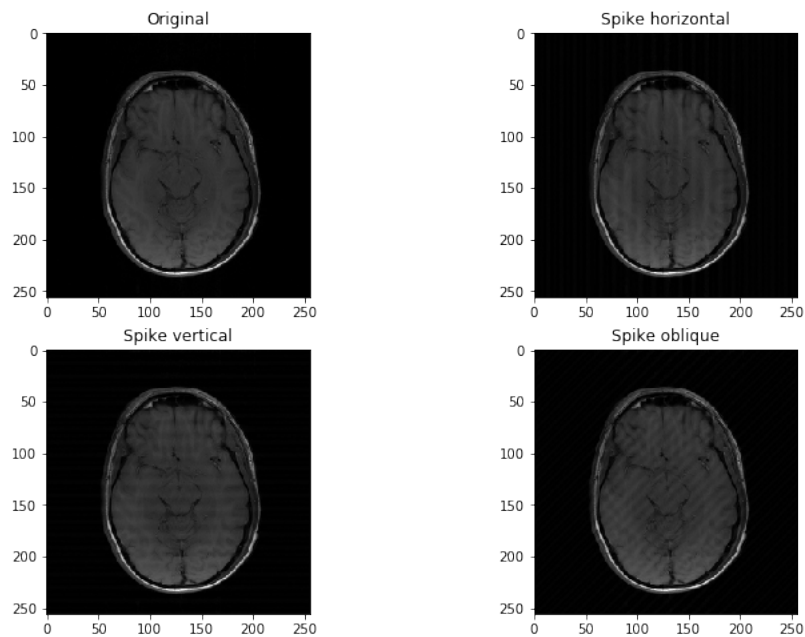
# Spike vertical
k_v = add_spike(k_v, (cx+20, cy))
k_v = add_spike(k_v, (cx-20, cy))

# Spike oblique
k_o = add_spike(k_o, (cx+20, cy+20))
k_o = add_spike(k_o, (cx-20, cy-20))

# IFFT
img_h = np.abs(np.fft.ifft2(np.fft.ifftshift(k_h)))
img_v = np.abs(np.fft.ifft2(np.fft.ifftshift(k_v)))
img_o = np.abs(np.fft.ifft2(np.fft.ifftshift(k_o)))

plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.imshow(image, cmap='gray')
plt.title("Original")
plt.subplot(2,2,2)
plt.imshow(img_h, cmap='gray')
plt.title("Spike horizontal")
plt.subplot(2,2,3)
plt.imshow(img_v, cmap='gray')
plt.title("Spike vertical")
plt.subplot(2,2,4)
plt.imshow(img_o, cmap='gray')
plt.title("Spike oblique")
plt.show()

```



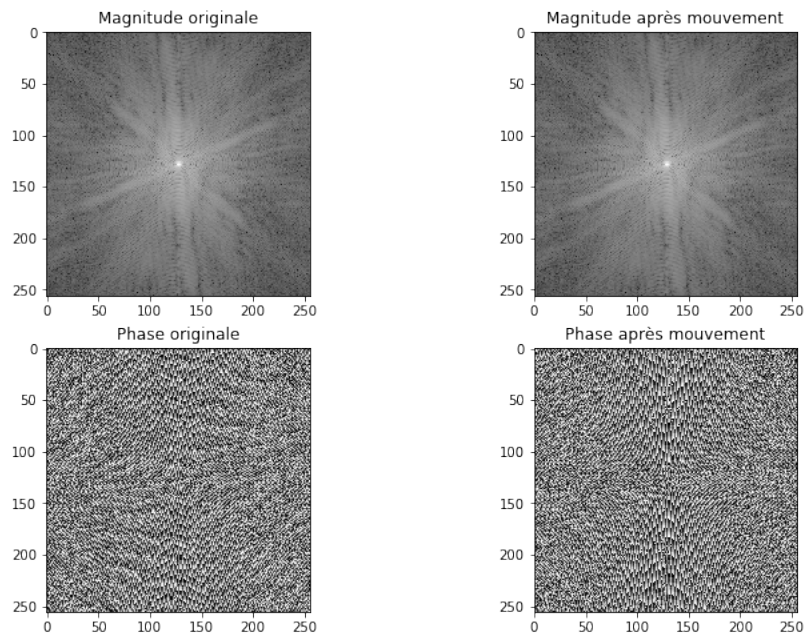
1.8. Artefact de mouvement. Toujours à partir de la même image, observez l'effet d'un mouvement de l'image sur sa transformée de Fourier : - Décalez votre image de 10 pixels (fonction `np.roll` dont vous trouverez la documentation [ici] (<https://docs.scipy.org/doc/numpy/reference/generated/numpy.roll.html>)). - Calculez les k-space de l'image originale et de l'image décalée. - Affichez la magnitude et la phase du k-space avant et après le mouvement. Observez-vous une différence sur la magnitude du k-space? Sur la phase du k-space? Expliquez à quoi sont dus ces résultats. Indiquez aussi pourquoi la différence de phase du k-space (entre l'image originale et l'image décalée) à une allure en "dents de scie". (2pts)

```
In [22]: # Décaler l'image de 10 pixels
image_shift = np.roll(image, shift=10, axis=0) # déplacement vertical

# FFT des deux images
k_original = np.fft.fftshift(np.fft.fft2(image))
k_shifted = np.fft.fftshift(np.fft.fft2(image_shift))

# Magnitude et phase
mag_orig = np.abs(k_original)
mag_shift = np.abs(k_shifted)
phase_orig = np.angle(k_original)
phase_shift = np.angle(k_shifted)

plt.figure(figsize=(12,8))
plt.subplot(2,2,1)
plt.imshow(np.log(mag_orig+1), cmap='gray')
plt.title("Magnitude originale")
plt.subplot(2,2,2)
plt.imshow(np.log(mag_shift+1), cmap='gray')
plt.title("Magnitude après mouvement")
plt.subplot(2,2,3)
plt.imshow(phase_orig, cmap='gray')
plt.title("Phase originale")
plt.subplot(2,2,4)
plt.imshow(phase_shift, cmap='gray')
plt.title("Phase après mouvement")
plt.show()
```



Après avoir décalé l'image de 10 pixels, on observe que la magnitude du k-space reste pratiquement identique avant et après le mouvement. La structure globale est conservée, avec un point central très intense et des motifs similaires, ce qui s'explique par le fait qu'un déplacement spatial de l'image ne modifie pas le contenu fréquentiel mais uniquement sa phase. De même, la phase du k-space montre une structure globale similaire entre l'image originale et l'image déplacée, mais elle change de manière locale avec de légères différences dans la forme. Cela est dû au fait que la phase encode la position des structures et est donc sensible aux déplacements. Enfin, dans l'image après mouvement, on remarque que la forme globale reste la même mais que l'intensité visuelle semble légèrement différente, avec des zones un peu plus grises ou plus sombres. C'est lié aux effets numériques de reconstruction et à la manière dont la phase modifie localement les interférences lors de la transformée inverse, sans changer la structure globale de l'image.

1.9. Décrivez une procédure permettant de simuler un artefact correspondant à un mouvement global du patient en cours d'acquisition. (1pt) **Astuce: Pensez au domaine dans lequel le patient bouge et au domaine dans lequel on mesure notre signal en IRM.**

Pour simuler un artefact de mouvement global du patient, on doit tenir compte du fait que le patient bouge dans l'espace image (espace réel), alors que l'IRM enregistre le signal dans le k-space. Une façon de simuler cela est de modifier l'image pendant l'acquisition en la décalant (par exemple avec `np.roll()`) à différents moments, comme si le patient changeait légèrement de position pendant que les lignes du k-space sont acquises. Ensuite, on reconstruit l'image avec la transformée de Fourier inverse. Une autre façon équivalente est de simuler le mouvement directement dans le k-space en transformant des versions légèrement déplacées de l'image, ce qui correspond au fait que chaque ligne du k-space est acquise à des instants différents où le patient peut être dans une position différente. Ce décalage entre les différentes lignes du k-space crée des incohérences lors de la reconstruction, ce qui produit des artefacts dans l'image finale (flou, duplication ou ghosting).

1.10. Implémentez cette procédure pour un mouvement de 10 pixels dans la direction de votre choix. Considérez deux cas : - cas A: la deuxième moitié du k-space est affectée par le mouvement - cas B: une ligne sur deux est affectée par le mouvement. Expliquez vos résultats. (2pts)

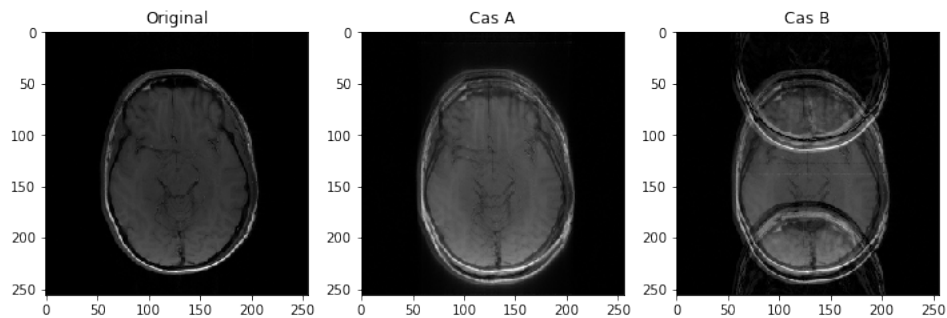
```
In [23]: kspace = np.fft.fftshift(np.fft.fft2(image))
h, w = kspace.shape

# image mouvement (10 pixels vertical)
image_shift = np.roll(image, shift=10, axis=0)
kspace_shift = np.fft.fftshift(np.fft.fft2(image_shift))

# CAS A : moitié du k-space affectée
k_A = kspace.copy()
k_A[h//2:, :] = kspace_shift[h//2:, :]
img_A = np.abs(np.fft.ifft2(np.fft.ifftshift(k_A)))

# CAS B : 1 ligne sur 2 affectée
k_B = kspace.copy()
k_B[:, ::2, :] = kspace_shift[:, ::2, :]
img_B = np.abs(np.fft.ifft2(np.fft.ifftshift(k_B)))

plt.figure(figsize=(12,6))
plt.subplot(1,3,1)
plt.imshow(image, cmap='gray')
plt.title("Original")
plt.subplot(1,3,2)
plt.imshow(img_A, cmap='gray')
plt.title("Cas A")
plt.subplot(1,3,3)
plt.imshow(img_B, cmap='gray')
plt.title("Cas B")
plt.show()
```



L'image originale est nette et bien définie. Dans le cas A, où seule la deuxième moitié du k-space provient d'une image déplacée, on observe une superposition de deux positions légèrement différentes du cerveau. Cela donne un effet de flou ou de double contour, surtout dans la direction du mouvement, car les basses fréquences et les hautes fréquences ne correspondent pas à la même position du patient. Dans le cas B, où une ligne sur deux du k-space est affectée par le mouvement, l'artefact est plus important. On retrouve l'effet du cas A, mais renforcé par une incohérence plus fréquente dans les données. Cela crée une duplication plus marquée de l'image, avec des structures qui semblent apparaître en haut et en bas, donnant un effet de répétition ou de ghosting similaire à un sous-échantillonnage dans le k-space.

Section 2 - IRM anatomique (5pts)

D'autres formats d'images sont souvent utilisés en IRM, par exemple le format NIfTI (.nii). Chargez l'image anatomique `anat/mprage.nii.gz` en utilisant `nib.load`.

```
In [7]: nii = nib.load('anat/mprage.nii.gz')
```


2.1. Il s'agit d'une image à 3 dimensions. Quelle est le nombre de voxels dans chacune des dimensions ? Quelle est la taille de chaque voxel (en mm) ? (1pt)
Indice : Regardez l'attribut 'pixdim' de nii.header

nib.load renvoie un objet nifti constitué d'un header et de l'image. Utilisez objet.header pour accéder au header et object.dataobj pour accéder au données d'image.

```
In [5]: data = nii.dataobj
header = nii.header

# Obtenir le nombre de voxels par axe
dimensions = data.shape
print("Nombre de voxels par axe:", dimensions)

# Obtenir la taille des voxels (en mm)
voxel_size = header['pixdim'][1:4]
print("Taille des voxels (mm):", voxel_size)
```

Nombre de voxels par axe: (256, 256, 128)
Taille des voxels (mm): [1. 1. 2.]
L'image IRM est en 3D avec 256 voxels en x, 256 en y et 128 en z. Chaque voxel a une taille de 1 mm en x, 1 mm en y et 2 mm en z. Cela signifie que l'image est plus précise dans les directions x et y que dans la direction z.

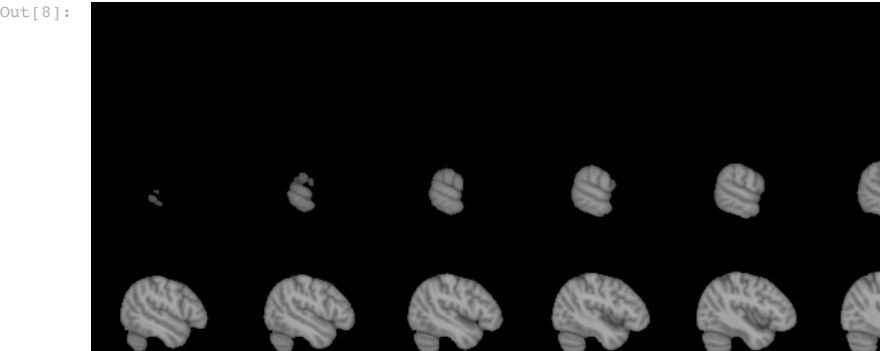
Visualisez l'image à l'aide de la cellule ci-dessous. Cette image a été prise avec une séquence 3D de type écho de gradient (gradient echo). Le TR est très faible (inférieur à 10ms), il s'agit donc d'une image pondérée en T1. Un pulse d'inversion-récupération prépare les spins afin d'augmenter le contraste entre les tissus (suppression du liquide cérébrospinal). Le nom de cette séquence est « MPRAGE ».

2.2. Faites une capture d'écran (screenshot) de la vue axiale (seulement la vue axiale). Quelle est cette structure noire située aux coordonnées (-10, -22, 28) ? Quelle stratégie a permis d'annuler le signal pour cette structure ? (2pts)

Vous pouvez vous déplacer sur l'image à l'aide de la souris. En cas de problème avec le viewer (l'image apparaît hachée), cliquez sur une des vues correctes et cela reviendra à la normale.

```
In [8]: # Visualiser l'image en utilisant nilearn plotting
nip.view_img(nii, cut_coords=[-10, -22, 28], title="Anatomical image", cmap="gray", symmetric_cmap=False)
```

/opt/anaconda3/envs/env-GBM8378/lib/python3.6/site-packages/nilearn/image/resampling.py:510: UserWarning: Casting data from int32 to float32
warnings.warn("Casting data from %s to %s" % (data.dtype.name, aux))



La
stru
obse
au
coc
(-10
-22
28)
cor
au
liq
cér

2.3.
A
quoi
pourrait
correspc
le
petit
blob
gris
aux
coordon
(56,68,2

Le
petit
blob
gris
pourra
corres
à
un
objet
extern
utilisé
comme
marqu

2.4.
Observe
la
structur
très
brillante
aux
coordon
(-2,-6,-'
Il
s'agit
de

La
structu
très
brillant
corres
à
l'artère
basilair
qui
contier
du
sang
en

Section 3 - IRM fonctionn (15pts)

Nous
allons
étudier
l'image
'fmri.nii'
La
particularit
de ce
fichier
est
qu'il
s'agit

```
In [9]: # Affi  
# Note  
# les  
fmri_w  
fmri_w  
<Figure
```

dar	?	de	l'artère	mouvement	d'une
le	(1pts)	repère	basilaire	Elle	image
ver		(fiduciel)	Pourquoi	n'apparaît	4D
laté		Ce	le	pas	(versus
Dar		sont	signal	noire	3D
cet		de	de	comme	pour
image		petits	celle-ci	le	l'image
MP		objets	n'a	liquide	anatomique
(pour		qu'on	pas	cérébral	Les
en		place	été	parce	dimensions
T1)		sur	supprimé	que	(x,y,z,t)
une		la	comme	les	de
technique		tête	la	technique	l'image
d'ir		ou	structurale	d'IRM	sont :
résumé		sur	noire	peuvent	64x64x210
est		la	en	"annuler"	Cela
utilisé		peau	2.2	le	implique
pour		pendant	?	signal	qu'il y
supprimer		une	(1pts)	des	a 180
le		IRM		liquide	volumes
signal		pour		immobilité	3D,
du		servir		comme	un
LCR		de		le	volume
Ca		point		LCR,	étant
signal		de		mais	mesuré
qu'		référer		pas	toutes
un		et		celui	les
mo		aider		du	TR
pré		à		sang	secondes.
de		se		qui	Ici, le
la		repère		circule	temps
séquence		ou		rapidement	de
le		à		Comme	répétition
signal		aligner		le	TR =
du		les		sang	3
liquide		images		est	secondes
des		du		toujours	. La
0,		cerveau		en	séquence
dorsal		et		mouvement	utilisée
il		du		de	pour
apparaître		crâne.		nouvel	acquérir
noir				particulier	ces
sur				entrent	volumes
l'ir				dans	est
ce				l'image	une
qui				et	séquence
per				donner	en
de				un	écho
mise				signal	de
voilà				fort,	gradient,
les				ce	pondérée
aut				qui	en
struc				rend	T2*,
du				l'artère	utilisant
cer				brillant	la
				À	technique
				l'invers	echo
				le	planar
				liquide	imaging
				cérébral	(EPI).
				est	L'intérêt
				plus	d'utiliser

stable
et
peut
être
supprimer
par
les
réglages
de
l'IRM,
donc
il
apparaît
noir.

la
technique
EPI
est
d'obtenir
un
volume
du
cerveau
complet
très
rapidement.
Ici,
chaque
volume
est
acquis
en
seulement
3
secondes
!! Par
comparaison
le
volume
anatomique
précédent
a été
acquis
en 5
minutes.
Bien
sûr, la
qualité
d'image
est
moindre.
Affichons
l'image:

